

A binary tree is a data structure where each node has at most a left and right node. The tree in this lab is stored inside a Python list, not node objects. Each position index represents a node. If a node is at index i , its left child is at $2*i + 1$ and its right child is at $2*i + 2$. and missing children stored as None placeholders so the structure stays aligned. So, traversal uses index math, rather than pointers.

A big challenge is that there are no real links between nodes. That makes deletion harder, as you cannot reconnect pointers. I handled it by using the formula above and implementing three deletion cases. If the node has no children, set its value to `None`. If it has exactly one child, copy the child value up and keep pushing the empty slot down until I reach a leaf. If it has two children, replace the node's value with its inorder successor (the leftmost node in the right subtree) and then delete the successor node. This keeps things on a single downward path running in $O(h)$ where h represents the tree height.

As a possible improvement, I could track the minimum and maximum keys currently stored in the tree. Before doing a search or delete, if $k < \text{min_key}$ or $k > \text{max_key}$, I can immediately conclude the key isn't in the tree and avoid unnecessary traversal. I also believe GTA may prefer more helper functions to reduce redundancy.